NOAA Global Workflow Experiment Configuration System

A Comprehensive Technical Reference

Environmental Modeling Center
National Centers for Environmental Prediction
NOAA/NWS

EIB MCP-RAG Development Team

February 11, 2026
Version 1.0

Abstract

This document provides a comprehensive technical reference for the NOAA Global Workflow experiment configuration system. It details the complete lifecycle of experiment creation, from initial definition through YAML case files or command-line interface, through Jinja2 template rendering, to runtime configuration sourcing during job execution on High Performance Computing (HPC) platforms.

The document covers the application factory pattern used to register six distinct application types (GFS, GEFS, SFS, GCAFS in cycled and forecast-only modes), the hierarchical configuration file system with 105+ config files including 16 Jinja2 templates, platform-specific resource allocation for seven HPC platforms, and the Rocoto workflow engine integration.

This reference is intended for developers, operators, system administrators, and stakeholders seeking to understand the internal architecture of the Global Workflow experiment configuration system.

Contents

1 Introduction

1.1 Purpose and Scope

The NOAA Global Workflow is the operational infrastructure for numerical weather prediction (NWP) at the National Centers for Environmental Prediction (NCEP). This system produces the Global Forecast System (GFS) and related ensemble and coupled model forecasts that support weather prediction worldwide.

This document provides a complete technical reference for understanding how experiments are configured and instantiated within the Global Workflow. An **experiment** in this context refers to a specific configuration of the workflow for a particular purpose, such as:

- Operational production forecasts
- Development and testing
- Continuous integration validation
- Retrospective analyses
- Research experiments

1.2 Target Audience

This document is intended for:

Developers Writing new components or modifying existing workflow functionality

Operators Running and monitoring workflow experiments on HPC systems

System Administrators Managing infrastructure and resource allocation

New Team Members Onboarding to understand the system architecture

Stakeholders Requiring technical understanding of the system

1.3 Document Conventions

- `Monospace text` indicates file names, paths, commands, or code
- **Bold text** indicates important terms or emphasis
- Blue text indicates hyperlinks
- Code blocks include syntax highlighting for clarity
- Diagrams use consistent color coding: blue for Python, orange for Bash, green for YAML

1.4 System Overview

The Global Workflow experiment configuration system operates through three distinct phases:

1. **Experiment Definition** — Specification of parameters via YAML or CLI
2. **Experiment Setup** — Directory creation and template rendering
3. **Runtime Configuration** — Dynamic sourcing during job execution

Figure 1 illustrates this three-phase architecture.

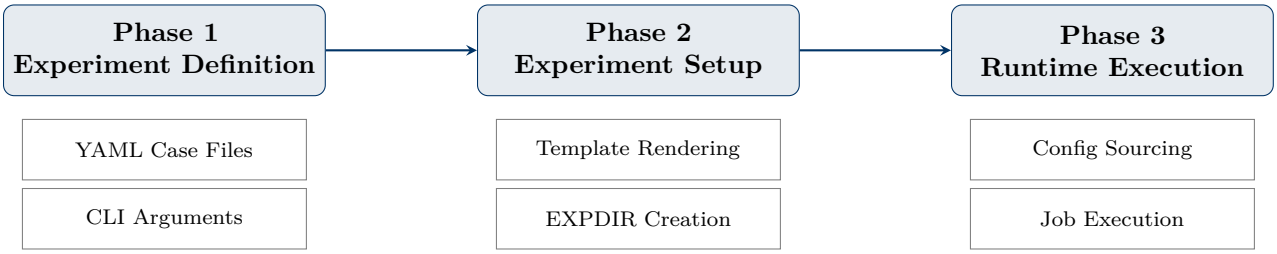

Figure 1: High-Level Overview of Experiment Configuration Phases

2 System Architecture Overview

2.1 Directory Structure

The Global Workflow follows a standardized directory structure organized by function:

Global Workflow Directory Structure

```
global-workflow/
          dev/                        # Development directory
                ci/                     # Continuous integration
                      cases/              # YAML case files for CI testing
                jobs/                   # J-job scripts (entry points)
                parm/                   # Parameter and config files
                      config/             # Configuration by network (gfs,
   gcafs, etc.)
                workflow/               # Python workflow infrastructure
                    applications/       # Application factory
                    rocoto/             # Rocoto workflow generation
          scripts/                    # Execution scripts (ex*.sh)
          ush/                        # Utility scripts
          env/                        # Platform environment files
          parm/                       # Shared parameter files
          sorc/                       # Source code and build
```

2.2 Component Relationships

Figure 2 shows the relationships between major system components.

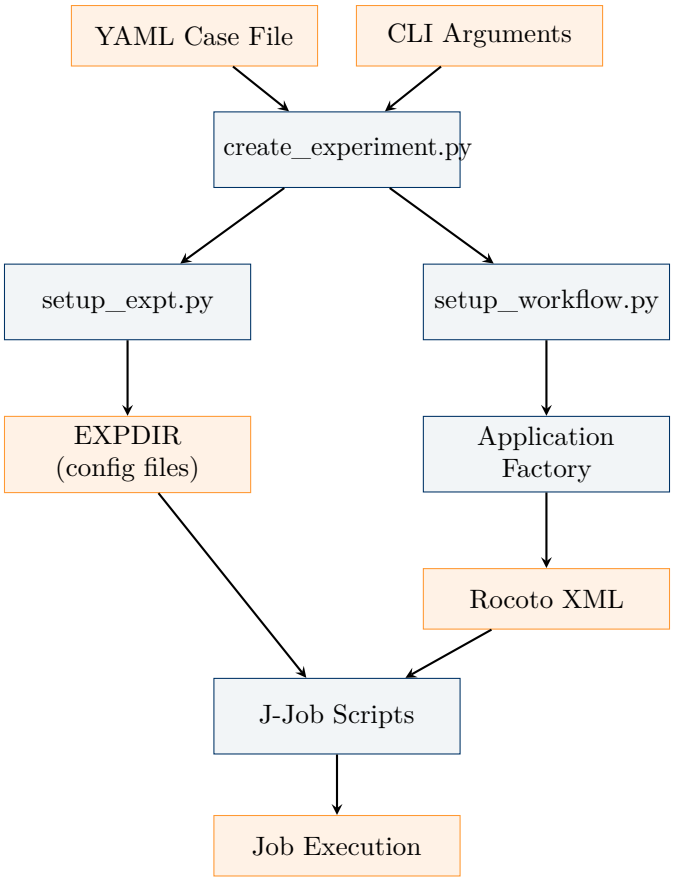

Figure 2: Component Relationship Diagram

2.3 Data Flow Overview

1. **Input**: User provides experiment specification (YAML or CLI)
2. **Processing**: Python scripts create directories and render templates
3. **Generation**: Application factory produces Rocoto workflow XML
4. **Execution**: Rocoto orchestrates job submission and dependency management
5. **Runtime**: J-jobs source configuration and execute scripts

3 Application Factory Pattern

3.1 Design Pattern Overview

The Global Workflow uses the **Factory Pattern** to instantiate application configurations dynamically based on the network type (NET) and operational mode (MODE). This design enables:

- **Extensibility**: New application types can be added without modifying core logic
- **Encapsulation**: Each application manages its own task definitions
- **Consistency**: Uniform interface across all application types

3.2 Registered Applications

The factory registers six application configurations:

Table 1: Registered Application Configurations

Registration Key	NET	MODE	Class
`gfs_cycled`	GFS	Cycled	GFSCycledAppConfig
`gfs_forecast-only`	GFS	Forecast-only	GFSForecastOnlyAppConfig
`gefs_forecast-only`	GEFS	Forecast-only	GEFSAppConfig
`sfs_forecast-only`	SFS	Forecast-only	SFSAppConfig
`gcafs_forecast-only`	GCAFS	Forecast-only	GCAFSForecastOnlyAppConfig
`gcafs_cycled`	GCAFS	Cycled	GCAFSCycledAppConfig

3.3 Factory Implementation

application_factory.py

```python
from wxflow import Factory
from applications.gfs_cycled import GFSCycledAppConfig
from applications.gfs_forecast_only import GFSForecastOnlyAppConfig
from applications.gefs import GEFSAppConfig
from applications.sfs import SFSAppConfig
from applications.gcafs_forecast_only import GCAFSForecastOnlyAppConfig
from applications.gcafs_cycled import GCAFSCycledAppConfig

# Initialize the application configuration factory
app_config_factory = Factory('AppConfig')

# Register application configurations
app_config_factory.register('gfs_cycled', GFSCycledAppConfig)
app_config_factory.register('gfs_forecast-only', GFSForecastOnlyAppConfig)
app_config_factory.register('gefs_forecast-only', GEFSAppConfig)
app_config_factory.register('sfs_forecast-only', SFSAppConfig)
app_config_factory.register('gcafs_forecast-only',
    GCAFSForecastOnlyAppConfig)
app_config_factory.register('gcafs_cycled', GCAFSCycledAppConfig)
```

3.4 AppConfig Base Class

All application configurations inherit from the `AppConfig` base class, which defines the interface for task generation:

applications.py (Base Class)

```python
from abc import ABC, abstractmethod

class AppConfig(ABC):
    """Base configuration class for all applications."""

    def __init__(self, conf):
        self.conf = conf
```

```
        self._base = conf.parse_config('config.base')

    @abstractmethod
    def get_task_names(self) -> dict:
        """Return dictionary of task names by run type.

        Returns
        -------
        dict
            Keys are run types (e.g., 'gfs', 'gdas')
            Values are lists of task names in execution order
        """
        pass
```

3.5 Task Name Generation Example

Each application defines its tasks and their order:

gfs_cycled.py (Excerpt)

```
class GFSCycledAppConfig(AppConfig):

    def get_task_names(self) -> dict:
        task_names = {
            'gdas': ['prep', 'anal', 'sfcanl', 'analcalc',
                     'fcst', 'upp', 'atmos_products'],
            'gfs':  ['fetch', 'prep', 'atmanlinit', 'atmanlvar',
                     'atmanlfv3inc', 'atmanlfinal', 'sfcanl',
                     'analcalc', 'fcst', 'upp', 'atmos_products',
                     'tracker', 'genesis', 'arch']
        }

        # Add optional tasks based on configuration
        if self._base.get('DO_WAVE', 'NO') == 'YES':
            for run in task_names:
                task_names[run].extend(['waveinit', 'waveprep',
                                        'wavepostsbs', 'wavepostpnt'])

        return task_names
```

4 Experiment Definition

4.1 Two Methods of Experiment Definition

Experiments can be defined through two equivalent methods:

Table 2: Experiment Definition Methods

Method	Use Case	Entry Point
YAML Case File	CI testing, reproducible configs	`create_experiment.py -y file.yaml`
CLI Arguments	Interactive/manual setup	`setup_expt.py NET MODE -args`

4.2 YAML Case File Structure

YAML case files provide a complete experiment specification:

Example: C48_S2SW.yaml

```yaml
experiment:
  net: gfs
  mode: forecast-only
  pslot: {{ 'pslot' | getenv }}
  app: S2SW
  resdetatmos: 48
  resdetocean: 5.0
  comroot: {{ 'RUNTESTS' | getenv }}/COMROOT
  expdir: {{ 'RUNTESTS' | getenv }}/EXPDIR
  idate: 2021032312
  edate: 2021032312
  yaml: {{ HOMEgfs }}/dev/ci/cases/yamls/gfs_defaults_ci.yaml

workflow:
  engine: rocoto
  rocoto:
    maxtries: 2
    cyclethrottle: 3
    taskthrottle: 25
    verbosity: 2
```

4.3 Key Experiment Parameters

Table 3: Experiment Definition Parameters

Parameter	Example	Description
net	gfs	Network type (gfs, gefs, gcafs, sfs)
mode	cycled	Operational mode (cycled, forecast-only)
pslot	C384_S2SW	Experiment slot name
app	S2SW	Application type (ATM, S2S, S2SW, S2SWA)
resdetatmos	384	Atmospheric resolution (becomes C384)
resdetocean	0.25	Ocean resolution (becomes 025)
idate	2021032312	Initial date (YYYYMMDDHH)
edate	2021033112	End date (YYYYMMDDHH)
nens	20	Number of ensemble members
start	cold	Start type (cold, warm)

4.4 CLI-Based Experiment Setup

CLI Experiment Setup

```
# GFS Cycled experiment
python setup_expt.py gfs cycled \
    --pslot C384_S2SW \
    --app S2SW \
    --resdetatmos 384 \
    --resdetocean 0.25 \
    --idate 2024010100 \
    --edate 2024011000 \
    --expdir /scratch/expdir \
    --comroot /scratch/comroot

# GEFS Forecast-only experiment
python setup_expt.py gefs forecast-only \
    --pslot C384_GEFS_20mem \
    --nens 20 \
    --resdetatmos 384 \
    --idate 2024010100
```

4.5 CI Case File Categories

The CI system organizes case files by purpose:

Table 4: CI Case File Categories

Directory	Count	Purpose
`pr/`	20	Pull request tests (run on every PR)
`gfsv17/`	20	GFS v17 production-like configurations
`gcafsv1/`	6	GCAFS v1 tests
`sfsv1/`	2	SFS v1 tests
`hires/`	2	High-resolution tests (C768, C1152)
`weekly/`	2	Weekly extended tests
`yamls/`	19	Default settings (included by cases)

5 Configuration File System

5.1 Configuration File Categories

The GFS configuration directory contains 105 files organized into categories:

Table 5: Configuration File Statistics

Category	Count
Jinja2 Templates (`*.j2`)	16
Plain Bash Configs	88
YAML Defaults Directory	1
Total	105

5.2 Jinja2 Template Files

Templates are rendered at setup time with experiment-specific values:

Table 6: Jinja2 Template Files

Template	Purpose
`config.base.j2`	Global settings (machine, paths, toggles)
`config.fcst.j2`	Forecast configuration
`config.atmanl.j2`	Atmospheric analysis
`config.atmensanl.j2`	Ensemble atmospheric analysis
`config.marineanl.j2`	Marine analysis
`config.marinebmat.j2`	Marine background error matrix
`config.marineanlecen.j2`	Marine ensemble centering
`config.marineanlletkf.j2`	Marine LETKF analysis
`config.aeroanl.j2`	Aerosol analysis
`config.aero.j2`	Aerosol configuration
`config.prep.j2`	Data preparation
`config.prepoceanobs.j2`	Ocean observation preparation
`config.snowanl.j2`	Snow analysis
`config.esnowanl.j2`	Ensemble snow analysis
`config.ocn.j2`	Ocean configuration
`config.stage_ic.j2`	Initial condition staging

5.3 Template Rendering Example

config.base.j2 Template (Excerpt)

```
#! /usr/bin/env bash

########## config.base ##########
# Common to all steps

echo "BEGIN: config.base"

# Machine environment
export machine="{{ MACHINE }}"

# Account, queue, etc.
export ACCOUNT="{{ ACCOUNT }}"
```

```
export QUEUE="{{ QUEUE }}"
export PARTITION_BATCH="{{ PARTITION_BATCH }}"

# Experiment settings
export PSLOT="{{ PSLOT }}"
export CASE="{{ CASE_CTL }}"

# Directories
export HOMEgfs="{{ HOMEgfs }}"
export EXPDIR="{{ EXPDIR }}"
export COMROOT="{{ COMROOT }}"
```

After rendering with context `{MACHINE: "HERA", PSLOT: "C384_S2SW", CASE_CTL: "C384"}`:

Rendered config.base

```
#! /usr/bin/env bash

########## config.base ##########
# Common to all steps

echo "BEGIN: config.base"

# Machine environment
export machine="HERA"

# Account, queue, etc.
export ACCOUNT="da-cpu"
export QUEUE="batch"
export PARTITION_BATCH=""

# Experiment settings
export PSLOT="C384_S2SW"
export CASE="C384"

# Directories
export HOMEgfs="/home/user/global-workflow"
export EXPDIR="/scratch/expdir/C384_S2SW"
export COMROOT="/scratch/comroot"
```

5.4 Hierarchical Config Structure

Configuration files follow a hierarchical inheritance pattern:

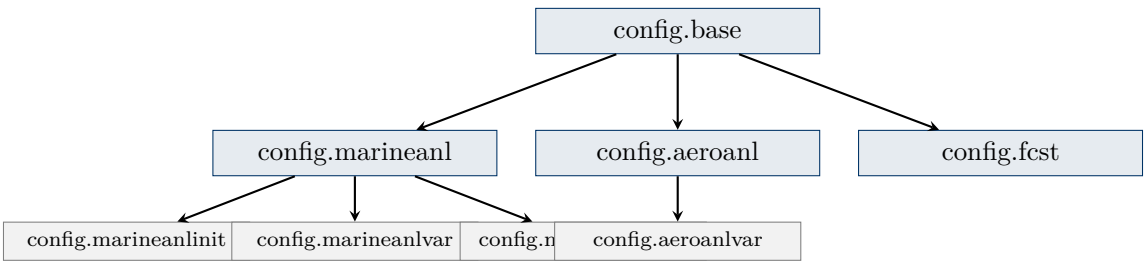

Figure 3: Hierarchical Configuration Inheritance

6 Platform Resource Configuration

6.1 Three-Layer Resource Architecture

Resource allocation follows a three-layer architecture that enables platform-independent experiment definitions while supporting platform-specific optimizations:

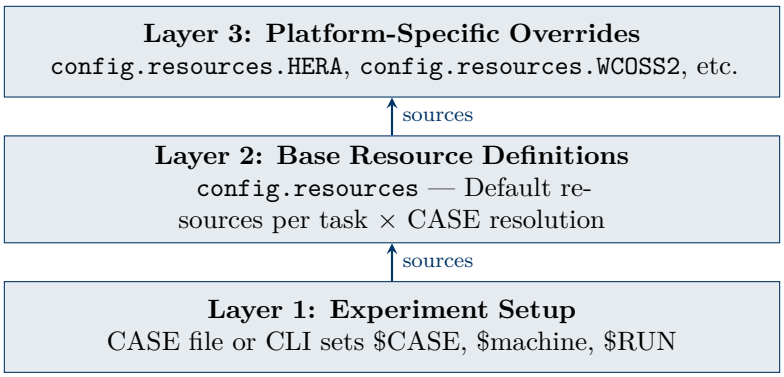

Figure 4: Three-Layer Resource Configuration Architecture

6.2 Platform Physical Limits

Each HPC platform has fixed physical constraints:

Table 7: HPC Platform Physical Limits

Platform	Tasks/Node	Memory/Node	Notes
WCOSS2	128	500 GB	Production (reservable: 500GB)
HERA	40	96 GB	R&D primary
HERCULES	80	500 GB	R&D
ORION	40	180 GB	R&D
GAEA C5	128	251 GB	Climate R&D
GAEA C6	192	384 GB	Climate R&D
URSA	192	360 GB	Cloud

6.3 Resource Definition by Task and Resolution

The `config.resources` script computes resources based on task and CASE:

config.resources (Excerpt)

```
case ${step} in
  "anal")
    if [[ "${RUN}" = *gdas ]]; then
      walltime="01:20:00"
      case ${CASE} in
        "C1152" | "C768")
          ntasks=780
          threads_per_task=5
          walltime="01:40:00"
          ;;
```

```
        "C384" | "C192")
          ntasks=160
          threads_per_task=10
          ;;
        "C96" | "C48")
          ntasks=84
          threads_per_task=5
          ;;
      esac
    fi
    tasks_per_node=$(( max_tasks_per_node / threads_per_task ))
    is_exclusive=True
    ;;
esac
```

6.4 Platform Override Example

config.resources.HERA

```
case ${step} in
  "prep")
    # Run on fewer tasks per node for memory requirement
    tasks_per_node=2
    ;;

  "eupd")
    case "${CASE}" in
      "C384")
        export ntasks=80  # Hera-specific reduction
        ;;
    esac
    export tasks_per_node=$(( max_tasks_per_node / threads_per_task ))
    ;;
esac
```

6.5 Complete Resource Matrix

Table 8: Resource Allocation by Task and Resolution (Selected Tasks)

Task	CASE	ntasks	threads	walltime	memory
anal	C768	780	5	01:40:00	exclusive
anal	C384	160	10	01:20:00	exclusive
anal	C96	84	5	01:20:00	exclusive
fcst (gfs)	C768	varies	varies	06:00:00	exclusive
fcst (gfs)	C384	varies	varies	06:00:00	exclusive
fcst (gdas)	C384	varies	varies	00:30:00	exclusive
upp	C1152	200	1	00:15:00	max_node
upp	C768	120	1	00:15:00	max_node
upp	C48	48	1	00:15:00	—
eupd	C768	480	6	00:45:00	exclusive
eupd	C384	270	8	00:45:00	exclusive

7 Experiment Directory Population

7.1 create_experiment.py Workflow

The `create_experiment.py` script orchestrates the complete setup:

create_experiment.py (Simplified)

```python
if __name__ == '__main__':
    user_inputs = input_args()

    # Parse YAML with Jinja2 templating
    data = AttrDict(HOMEgfs=_top)
    data.update(os.environ)
    testconf = parse_j2yaml(path=user_inputs.yaml, data=data)

    # Build arguments for setup_expt.py
    setup_expt_args = [testconf.experiment.net,
                       testconf.experiment.mode]
    for kk, vv in testconf.experiment.items():
        if kk not in ['net', 'mode']:
            setup_expt_args.extend([f"--{kk}", str(vv)])

    # Run setup_expt.py
    setup_expt.main(setup_expt_args)

    # Run setup_workflow.py
    experiment_dir = Path(testconf.experiment.expdir) / \
                     Path(testconf.experiment.pslot)
    setup_workflow.main([str(experiment_dir), 'rocoto'])
```

7.2 Template Rendering Process

update_configs() Function

```python
def update_configs(host, inputs):
    """Copy and render config files to EXPDIR."""

    # Combine host info and user inputs
    host_plus_inputs = AttrDict(host.info, **inputs_dict)
    host_plus_inputs.HOMEgfs = _top
    host_plus_inputs.MACHINE = str(host).upper()

    # Process each file in config directory
    files = os.listdir(inputs.configdir)
    for file in files:
        if file.endswith('.j2'):  # Jinja2 template
            input_template = f'{inputs.configdir}/{file}'
            output_file = f'{inputs.expdir}/{inputs.pslot}/{file[:-3]}'

            # Render template
            Jinja(input_template, host_plus_inputs).save(output_file)
        else:  # Plain file - copy as-is
            shutil.copy(f'{inputs.configdir}/{file}',
                        f'{inputs.expdir}/{inputs.pslot}/{file}')
```

7.3 Resulting EXPDIR Structure

Experiment Directory Contents

```
$EXPDIR/$PSLOT/
          config.base           # Rendered from config.base.j2
          config.fcst           # Rendered from config.fcst.j2
          config.marineanl      # Rendered from config.marineanl.j2
          config.anal           # Copied as-is
          config.arch_tars      # Copied as-is
          config.resources      # Copied (sources platform file at runtime)
          config.resources.HERA     # Copied
          config.resources.WCOSS2   # Copied
          ... (90+ additional config files)
          yaml/                 # Default YAML files
```

8 Rocoto Workflow Generation

8.1 Rocoto Overview

Rocoto is the workflow automation tool used by the Global Workflow. It:

- Manages job dependencies and execution order
- Handles job submission to HPC batch schedulers
- Provides monitoring and recovery capabilities
- Generates XML-based workflow definitions

8.2 Task Definition in gfs_tasks.py

Each task method defines its job parameters and dependencies:

gfs_tasks.py (Task Definition Example)

```python
def fcst(self):
    """Define forecast task."""

    deps = []

    # Depend on staging IC
    deps.append(rocoto.add_dependency(
        dep_type='task',
        name=f'{self.run}_stage_ic'))

    # Depend on analysis
    deps.append(rocoto.add_dependency(
        dep_type='task',
        name=f'{self.run}_sfcanl'))

    # Optional wave dependency
    if self.app_config.do_wave:
        deps.append(rocoto.add_dependency(
            dep_type='task',
            name=f'{self.run}_waveprep'))

    dependencies = rocoto.create_dependency(dep=deps)

    resources = self.get_resource('fcst')
    task = rocoto.create_task(name=f'{self.run}_fcst',
                              resources=resources,
                              dependencies=dependencies)
    return task
```

8.3 Dependency Types

Table 9: Rocoto Dependency Types

Type	Description
`task`	Depends on another task completing
`metatask`	Depends on a group of tasks
`data`	Depends on file existence
`cycleexist`	Depends on previous cycle
`sh`	Depends on shell command result

8.4 Generated XML Structure

Rocoto XML (Simplified)

```xml
<?xml version="1.0"?>
```

```
<!DOCTYPE workflow [
  <!ENTITY EXPDIR "/scratch/expdir/C384_S2SW">
  <!ENTITY HOMEgfs "/home/user/global-workflow">
]>
<workflow realtime="F" scheduler="slurm">
  <cycledef>202401010000 202401100000 06:00:00</cycledef>

  <task name="gfs_prep">
    <jobname>jgfs_prep_@Y@m@d@H</jobname>
    <account>da-cpu</account>
    <queue>batch</queue>
    <walltime>00:30:00</walltime>
    <nodes>1:ppn=14</nodes>
    <command>&HOMEgfs;/dev/jobs/JGLOBAL_ATMOS_PREP</command>
    <dependency>
      <taskdep task="gfs_fetch"/>
    </dependency>
  </task>

  <task name="gfs_fcst">
    <dependency>
      <and>
        <taskdep task="gfs_sfcanl"/>
        <taskdep task="gfs_stage_ic"/>
      </and>
    </dependency>
  </task>

</workflow>
```

9 Runtime Configuration Sourcing

9.1 jjob_header.sh Mechanism

The `jjob_header.sh` script is the universal entry point for all J-jobs:

jjob_header.sh (Key Section)

```
##############################
# Source relevant config files
##############################
export EXPDIR="${EXPDIR:-${HOMEgfs}/dev/parm/config}"
for config in "${configs[@]:-''}"; do
    source "${EXPDIR}/config.${config}" && true
    export err=$?
    if [[ ${err} -ne 0 ]]; then
        err_exit "Unable to load config config.${config}"
    fi
done

##########################################
# Source machine runtime environment
##########################################
```

```
source "${HOMEgfs}/env/${machine}.env" "${env_job}" && true
```

9.2 J-Job Config Specification

Each J-job explicitly declares its required configs:

J-Job Examples

```
# JGLOBAL_FORECAST
source "${HOMEgfs}/ush/jjob_header.sh" -e "fcst" -c "base fcst"

# JGLOBAL_MARINE_ANALYSIS_VARIATIONAL
source "${HOMEgfs}/ush/jjob_header.sh" -e "marineanlvar" \
    -c "base marineanl marineanlvar"

# JGLOBAL_AERO_ANALYSIS_INITIALIZE
source "${HOMEgfs}/ush/jjob_header.sh" -e "aeroanlinit" \
    -c "base aeroanl aeroanlinit"
```

9.3 Config Sourcing Order

Table 10: Configuration Sourcing Order for Selected Jobs

J-Job	Configs Sourced (in order)
JGLOBAL_FORECAST	config.base → config.fcst
JGLOBAL_ATMOS_ANALYSIS	config.base → config.anal
JGLOBAL_MARINE_ANALYSIS_VARIATIONAL	config.base → config.marineanl → config.marineanlvar
JGLOBAL_WAVE_PREP	config.base → config.wave → config.waveprep

9.4 Environment File Loading

After configs, platform-specific modules are loaded:

env/HERA.env (Excerpt)

```
#!/bin/bash
# HERA environment setup

job=${1:-}

# Load base modules
module purge
module use ${HOMEgfs}/modulefiles
module load gfs_v17.0.0

# Job-specific modules
case ${job} in
  "fcst")
    module load ufs_model
    ;;
```

```
  "anal")
    module load gsi
    ;;
esac

export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:${NETCDF}/lib
```

10 Complete Worked Example

10.1 Scenario: C384 S2SW Experiment on Hera

This section traces the complete flow from experiment definition to job execution.

10.1.1 Step 1: Define Experiment

Create Experiment

```
cd /home/user/global-workflow

python dev/workflow/create_experiment.py \
    -y dev/ci/cases/pr/C384_S2SW.yaml
```

10.1.2 Step 2: Examine YAML Case File

C384_S2SW.yaml

```
experiment:
  net: gfs
  mode: forecast-only
  pslot: C384_S2SW
  app: S2SW
  resdetatmos: 384
  resdetocean: 0.25
  idate: 2024010100
  edate: 2024010112
  yaml: dev/ci/cases/yamls/gfs_defaults_ci.yaml

workflow:
  engine: rocoto
  rocoto:
    maxtries: 2
    cyclethrottle: 3
    taskthrottle: 25
```

10.1.3 Step 3: Template Rendering

The setup process renders `config.base.j2` with:

Rendering Context

```
context = {
```

```
    'MACHINE': 'HERA',
    'PSLOT': 'C384_S2SW',
    'CASE_CTL': 'C384',
    'OCNRES': '025',
    'HOMEgfs': '/home/user/global-workflow',
    'EXPDIR': '/scratch/expdir',
    'COMROOT': '/scratch/comroot',
    'ACCOUNT': 'da-cpu',
    'QUEUE': 'batch',
    ...
}
```

10.1.4 Step 4: Application Factory Selection

Factory Selection

```
# setup_workflow.py determines app key
app_key = f"{net}_{mode}"  # "gfs_forecast-only"

# Factory returns appropriate config class
app_config = app_config_factory.create(app_key, conf)
# Returns: GFSForecastOnlyAppConfig instance
```

10.1.5 Step 5: Workflow XML Generation

The factory generates Rocoto XML with tasks in order:

Generated Task Order

```
gfs_fetch
gfs_prep
gfs_atmanlinit
gfs_atmanlvar
gfs_atmanlfv3inc
gfs_atmanlfinal
gfs_sfcanl
gfs_fcst
gfs_upp
gfs_atmos_products
gfs_waveinit      # S2SW includes wave
gfs_waveprep
gfs_wavepostsbs
gfs_arch
```

10.1.6 Step 6: Job Execution

When Rocoto submits `gfs_fcst`:

Runtime Execution Flow

```
# 1. Scheduler launches job on Hera
sbatch --account=da-cpu --partition=batch ...

```

```
# 2. JGLOBAL_FORECAST starts
#! /usr/bin/env bash
source "${HOMEgfs}/ush/jjob_header.sh" -e "fcst" -c "base fcst"

# 3. jjob_header.sh sources configs
source ${EXPDIR}/config.base     # Sets CASE=C384, machine=HERA
source ${EXPDIR}/config.fcst     # Sets forecast parameters

# 4. Machine environment loaded
source ${HOMEgfs}/env/HERA.env fcst

# 5. Execute script
${HOMEgfs}/scripts/exglobal_forecast.sh
```

10.1.7 Step 7: Resource Allocation

For the forecast task on Hera with C384:

Resource Resolution

```
# config.resources sources with step="fcst"
# Computes based on CASE=C384, machine=HERA

ntasks = 720           # Total MPI tasks
threads_per_task = 2   # OpenMP threads
tasks_per_node = 20    # Limited by Hera's 40 cores/node
walltime = "06:00:00" # GFS forecast walltime
memory = exclusive     # Full node allocation
```

A Complete Configuration File Inventory

Table 11: Complete GFS Configuration File Inventory

File	Purpose
config.base.j2	Global experiment settings (template)
config.fcst.j2	Forecast configuration (template)
config.atmanl.j2	Atmospheric analysis (template)
config.marineanl.j2	Marine analysis (template)
config.aeroanl.j2	Aerosol analysis (template)
config.resources	Base resource definitions
config.resources.HERA	Hera-specific overrides
config.resources.WCOSS2	WCOSS2-specific overrides
config.resources.ORION	Orion-specific overrides
config.resources.HERCULES	Hercules-specific overrides
config.anal	GSI analysis settings
config.sfcanl	Surface analysis settings
config.analcalc	Analysis calculation settings
config.upp	Unified Post Processor settings

Continued on next page

Continued from previous page

File	Purpose
config.atmos_products	Atmospheric product generation
config.wave	Wave model settings
config.waveinit	Wave initialization
config.waveprep	Wave preparation
config.tracker	Tropical cyclone tracker
config.genesis	Genesis tracking
config.arch_tars	Archive tarball settings
config.cleanup	Cleanup job settings
config.com	COM directory templates

B Platform Resource Reference

Table 12: Complete Platform Reference

Platform	Scheduler	Partition	Cores	Memory	Account
WCOSS2	PBS	—	128	500GB	GFS-DEV
HERA	Slurm	hera	40	96GB	da-cpu
ORION	Slurm	orion	40	180GB	da-cpu
HERCULES	Slurm	hercules	80	500GB	da-cpu
GAEA C5	Slurm	c5	128	251GB	nggps_psd
GAEA C6	Slurm	c6	192	384GB	nggps_psd

C Glossary

APP
: Application type: ATM (atmosphere-only), S2S (subseasonal-to-seasonal), S2SW (with waves), S2SWA (with aerosols)

CASE
: Atmospheric resolution designation (e.g., C384 = cubed-sphere with 384 grid points per tile edge)

COMROOT
: Common directory root for output products

EXPDIR
: Experiment directory containing configuration files

GFS
: Global Forecast System - primary deterministic forecast

GEFS
: Global Ensemble Forecast System - ensemble predictions

GCAFS
: Global Climate and Air quality Forecasting System

HOMEgfs
Root directory of Global Workflow installation

J-Job
Job control script that sets up environment and calls execution script

MODE
Operational mode: cycled (data assimilation) or forecast-only

NET
Network identifier: gfs, gefs, gcafs, sfs

PSLOT
Parallel slot name - unique experiment identifier

Rocoto
Ruby-based workflow automation tool

SFS
Subseasonal Forecast System

References

1. NOAA Environmental Modeling Center, "Global Workflow User's Guide," 2024.
2. Rocoto Workflow Manager Documentation, https://christopherwharrop.github.io/rocoto/
3. EMC Verification Guidelines (EE2 Standards), NOAA/NWS, 2023.
4. wxflow Python Library Documentation, NOAA/EMC, 2024.

Document Version History

Version	Date	Changes
1.0	February 11, 2026	Initial comprehensive release